

Sistemas Operativos I

Más mecanismos de Sincronización



Objeto Concurrente

Algo de **tipo objeto** está definido por un conjunto de operaciones y una especificación que describe el correcto funcionamiento de los **objetos** de ese tipo.

La única forma de interactuar con un objeto es mediante sus operaciones.

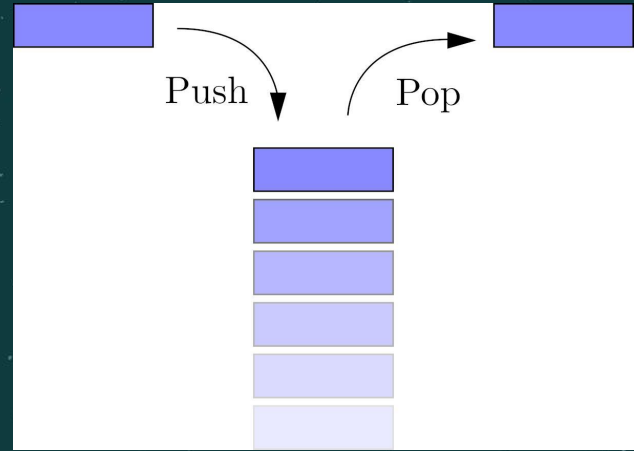
Un **objeto concurrente** es un objeto que puede ser invocado concurrentemente por varios procesos.

Objeto : Stack sin límite

- push
- pop

Como el stack no tiene límite siempre se puede invocar a la función `push`, y asumimos que hacer un `pop` de un stack vacío es `bottom`.

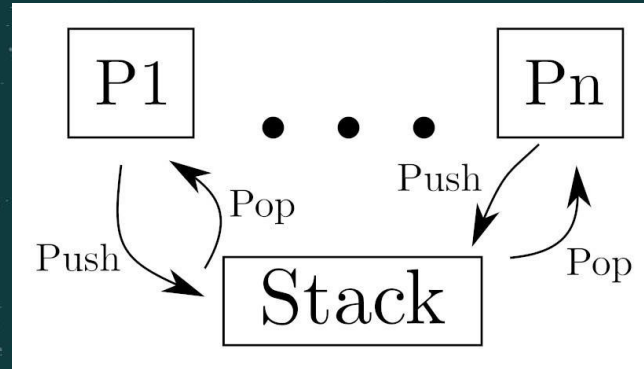
Especificación: todas las trazas que correspondan con una FILO (First-In-Last-Out).



Veamos una implementación de
un Stack sin límite

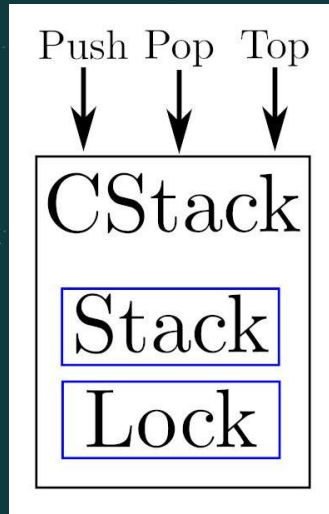
Objeto : Stack sin límite Concurrente

- `conc_push`
- `conc_pop`



Donde queremos que se comporte de la misma manera que el stack anterior pero será un stack compartido por varios procesos.

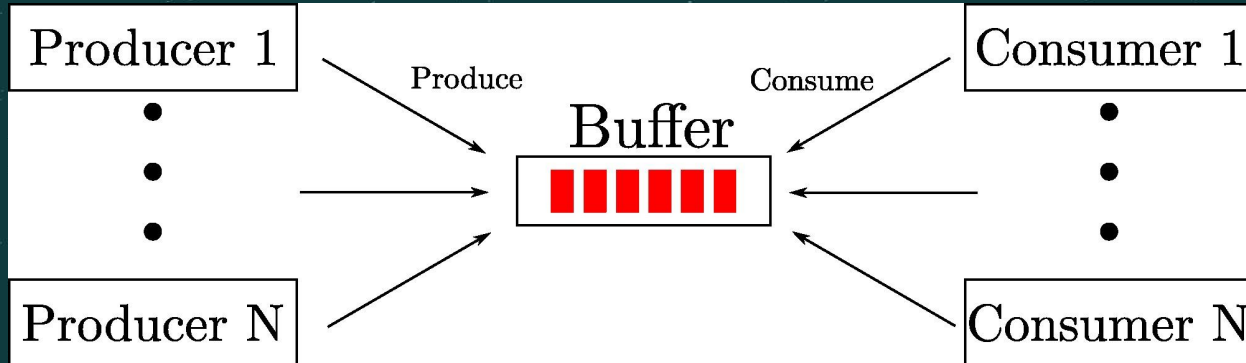
Objeto : Stack sin límite Concurrente



Implementar Stack sin límite
Concurrente

Problema Productor(es) Consumidor(es)

Asumamos dos tipos de procesos, aquellos que producen un elemento y otros que lo consumen. Asumimos que la producción y el consumo se pueden dar en diferentes momentos, y por ende introducimos un **buffer** de tamaño **K**.



Procesos!

Productores

```
while(true):  
    v <~ producir  
    buff.depositar(v)
```

Consumidores

```
while(true):  
    v <~ buff.tomar()  
    consumir(v)
```

Objeto Concurrente Buffer(I)

Asumimos que el objeto concurrente Buffer tiene un stack interno llamado **istack** de tamaño **K**.

`depositar(v)`

```
si hay lugar en istack:  
    stack_push(v)
```

`tomar()`

```
si no es vacio istack:  
    v = stack_pop()  
    retornar v
```

Objeto Concurrente Buffer(II)

Asumimos que el objeto concurrente Buffer tiene un stack **concurrente** interno llamado **icstack** de tamaño **K**.

`depositar(v)`

```
si hay lugar en icstack:  
    conc_stack_push(v)
```

`tomar()`

```
si no es vacio icstack:  
    v = conc_stack_pop()  
    retornar v
```

Objeto Concurrente Buffer(II')

Asumimos que el objeto concurrente Buffer tiene un stack interno llamado **istack** de tamaño **K** y un lock llamado **lock**.

`depositar(v)`

```
si hay lugar en icstack:  
    tomar(lock)  
    stack_push(v)  
    soltar(lock)
```

`tomar()`

```
si no esta vacio icstack:  
    tomar(lock)  
    v = stack_pop()  
    soltar(lock)  
    retornar v
```

Objeto Concurrente Buffer(III)

Asumimos que el objeto concurrente Buffer tiene un stack interno llamado **icstack** de tamaño **K** y un lock llamado **lock**.

`depositar(v)`

```
tomar(lock)
si hay lugar en icstack:
    stack_push(v)
soltar(lock)
```

`tomar()`

```
tomar(lock)
si no es vacio icstack:
    v = stack_pop()
soltar(lock)
retornar v
```

Objeto Concurrente Buffer(IV)

Asumimos que el objeto concurrente Buffer tiene un stack interno llamado **istack** de tamaño **K** y un lock llamado **lock**

depositar(v)

```
agregado = false
while(!agregado)
    tomar(lock)
    si hay lugar en icstack:
        stack_push(v)
        agregado = true
    soltar(lock)
```

tomar()

```
sacado = false
while (!sacado)
    tomar(lock)
    si no es vacio icstack:
        v = stack_pop()
        sacado = true
    soltar(lock)
retornar v
```



Variables de Condición



Variables de Condición!

Las variables de condición representan la idea que hay un conjunto de procesos que esperan cierta condición, y una vez que esta se cumple, se despiertan para que puedan volver a competir por los recursos.

Y se utilizan en conjunto con mutex lock.

- + **wait** : pone al hilo que la invoca a la espera de una condición liberando el lock que tiene.
- + **signal** : despierta a un hilo que esté a la espera
- + **broadcast** : despierta a todos los hilos esperando.



Variables de Condición!

```
#include <pthread.h>

int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_signal(pthread_cond_t *cond);

int pthread_cond_timedwait(pthread_cond_t *restrict cond, pthread_mutex_t
*restrict mutex, const struct timespec *restrict abstime);
int pthread_cond_wait(pthread_cond_t *restrict cond, pthread_mutex_t *restrict
mutex);
```

Objeto Concurrente Buffer Variables de Condición

depositar(v)

```
tomar(lock)
while (isFull())
    cond_wait(non_full_cond, lock)
stack_push(v)
cond_signal(non_empty_cond)
soltar(lock)
```

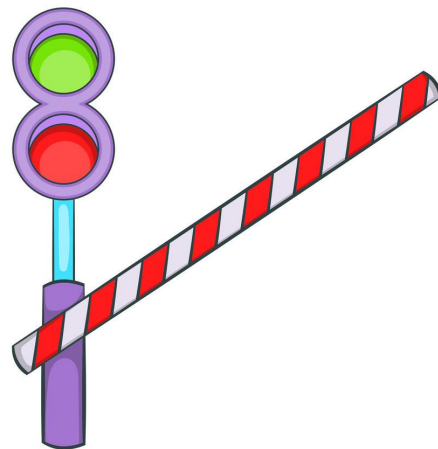
tomar()

```
tomar(lock)
while (isEmpty())
    cond_wait(non_empty_cond, lock)
v = stack_pop()
cond_signal(non_full_cond)
soltar(lock)
return v
```

Ejercicio: Productor - Consumidor con Variables de Condición



Semáforos





Semáforos!

Los semáforos son un objeto concurrente que representa la idea de un **contador concurrente**. Es decir, es un **contador** con dos operaciones **atómicas** *down* y *up*.

- Un semáforo S se inicia con un valor s_0 no negativo.
- El contador s_0 nunca se vuelve negativo
- *down* decrementa atómicamente en uno el contador
- *up* incrementa atómicamente en uno el contador

Invariante: $S = s_0 + \#(up) - \#(down)$



Semáforos!

Los semáforos son un poco más flexibles que los Mutex Locks.

- En principio podemos utilizarlo como lock (Semáforo Binario), inicializándolo en 1.
- Pero además podemos bloquear hasta que se haya completado una tarea (como un lock tomado), inicializándolo en 0.

Objeto Concurrente Buffer Semáforos

Asumimos que el objeto concurrente Buffer tiene un stack **concurrente** interno llamado **istack**, un semáforo **Lugares(K)** y **Ocupados(0)**.

`depositar(v)`

```
Lugares.down()  
conc_stack_push(v)  
Ocupados.up()
```

`tomar()`

```
Ocupados.down()  
t = conc_stack_pop()  
Lugares.up()  
retornar t
```



Semáforos!

```
#include <semaphore.h>
```

```
int sem_init(sem_t *sem, int pshared, unsigned int value);  
int sem_post(sem_t *sem);  
int sem_wait(sem_t *sem);  
int sem_trywait(sem_t *sem);  
int sem_timedwait(sem_t *sem, const struct timespec  
*abs_timeout);  
int sem_destroy(sem_t *sem);
```

Link with `-pthread`.

Ejercicio: Productor - Consumidor con Semáforos





Monitores



Monitores

```
typedef struct monitor {  
  
    shared_data;    // Shared data to be protected  
    sync_mechanism; // mutex, var. cond., semaphores  
} monitor_t;  
  
operation1(monitor_t *monitor){...}  
operation2(monitor_t *monitor){...}  
...
```

- En programación concurrente, un monitor es una estructura de sincronización que impide que los hilos accedan simultáneamente al estado de un objeto compartido y les permite esperar a que este cambie. Proporcionan un mecanismo para que los hilos renuncien temporalmente a un mutex, esperando a que se cumpla una condición, para luego recuperarlo y reanudar su tarea. Un monitor consta de un mutex (bloqueo) y al menos una variable de condición. Una variable de condición se "signalea" explícitamente cuando se modifica el estado del objeto, transfiriendo temporalmente el mutex a otro hilo que la espera.
- La característica que define a un monitor es que sus métodos se ejecutan con exclusión mutua: en cada momento, como máximo un hilo puede estar ejecutando cualquiera de sus métodos. Al usar una o más variables de condición, también se puede permitir que los threads esperen una condición determinada (de ahí el nombre "monitor"). **Un Monitor se dice que es un objeto concurrente thread-safe (seguro para múltiples threads).**



Barreras



Barreras

Una barrera es un punto de sincronización donde los hilos deben esperar hasta que un número específico de hilos haya llegado a ese punto antes de que todos puedan continuar.

```
#include <pthread.h>

int pthread_barrier_init(pthread_barrier_t *restrict barrier,
                        const pthread_barrierattr_t *restrict attr, unsigned count);
int pthread_barrier_wait(pthread_barrier_t *barrier);
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

Ejercicio: Barrera

Crear 3 threads y utilizar una barrera como punto de sincronización.